

Strukturni Dizajn Paterni

1 Adapter

Adapter je obrazac dizajna koji omogućava saradnju između različitih interfejsa. U našem sistemu za automatizaciju rezervisanja parking mjesta možemo koristiti Adapter za integraciju različitih vanjskih servisa za online plaćanje.

- Kreiranje interfejsa PaymentAdapter koji će definisati metode za obradu plaćanja unutar sistema.
- Implementacija konkretnih adaptera StripeAdapter i PayPalAdapter koji će prilagoditi interfejse vanjskih servisa interfejsu definisanom u sistemu.
- Unutar konkretnih adaptera moguće je implementirati komunikaciju prema API-ju vanjskih servisa za plaćanje.

Korištenje: Unutar sistema možemo koristiti PaymentAdapter za obradu plaćanja umjesto direktnog pozivanja vanjskih servisa. Na taj način sistem postaje nezavisan od konkretnih implementacija servisa za plaćanje, što olakšava buduće promjene i održavanje sistema.

2 Façade

Façade obrazac može se primijeniti kako bi se kreirao jednostavan interfejs za upravljanje procesom rezervacije parking mjesta. Ovaj interfejs će sakriti složenost podsistema i omogućiti korisnicima jednostavnije korištenje sistema.

Proces rezervacije uključuje više različitih klasa kao što su:

- ParkingObjekat
- Rezervacija
- Plaćanje
- Transakcija
- Notifikacija

Korištenje: Kreiranjem klase ParkingReservationFacade moguće je objediniti kompletnu logiku rezervacije parking mjesta. Korisnik sistema neće direktno komunicirati sa svim podsistemima, nego samo sa façade klasom koja će interno izvršavati provjeru dostupnosti parking mjesta, kreiranje rezervacije, obradu plaćanja i slanje notifikacija.

3 Decorator

Decorator je obrazac dizajna koji omogućava dinamičko dodavanje ili promjenu funkcionalnosti objektima. U našem sistemu možemo koristiti Decorator za proširenje funkcionalnosti korisnika prema njihovim ulogama.

- Definisanje interfejsa `UserFunctionality` koji predstavlja osnovne funkcionalnosti korisnika.
- Implementacija `BasicUserFunctionality` klase koja pruža osnovne funkcionalnosti korisnika.
- Kreiranje dekoratora kao što su `AdminFunctionalityDecorator` i `OwnerFunctionalityDecorator` koji dodaju dodatne mogućnosti korisnicima.

Korištenje: Nakon prijave korisnika moguće je dinamički dodijeliti odgovarajući dekorator prema ulozi korisnika. Na primjer, vlasnik parkinga može imati dodatne funkcionalnosti za upravljanje parking objektima, dok administrator može imati funkcionalnosti za pregled svih rezervacija i korisnika.

4 Proxy

Proxy obrazac može se koristiti za zaštitu osjetljivih podataka i kontrolu pristupa određenim funkcionalnostima sistema.

U sistemu za rezervaciju parking objekata određene funkcionalnosti trebaju biti dostupne samo administratorima ili vlasnicima parking objekata.

- Kreiranje klase `ParkingProxy` koja kontroliše pristup parking objektima.
- Provjera korisničkih prava prije izmjene podataka o parking objektima.
- Kontrola pristupa administrativnim funkcionalnostima sistema.

Korištenje: Proxy može provjeravati da li korisnik ima odgovarajuću ulogu prije nego što mu se dozvoli ažuriranje parking objekata, pregled svih transakcija ili upravljanje rezervacijama. Na ovaj način povećava se sigurnost sistema i štite osjetljivi podaci.

5 Bridge

Bridge obrazac omogućava odvajanje apstrakcije od implementacije kako bi se obje komponente mogle nezavisno razvijati.

U našem sistemu moguće je odvojiti način slanja notifikacija od same logike notifikacija.

- Kreiranje apstraktne klase `Notifikacija`.
- Kreiranje implementacionog interfejsa `NotificationSender`.

- Implementacija različitih načina slanja notifikacija kao što su EmailNotificationSender i InAppNotificationSender.

Korištenje: Sistem može koristiti različite načine slanja notifikacija bez izmjene osnovne klase Notifikacija. Na taj način moguće je jednostavno dodavati nove načine slanja obavještenja poput SMS ili push notifikacija.

6 Composite

Sistem za rezervaciju parking objekata sadrži više parking objekata koji mogu biti organizovani u grupe prema lokaciji, gradu ili tipu parkinga.

Composite obrazac omogućava grupisanje parking objekata u hijerarhijske strukture kako bi se omogućilo jednostavnije upravljanje većim skupovima parking objekata.

Korištenje: Implementacijom Composite obrasca moguće je kreirati grupe parking objekata koje sadrže pojedinačne parkinge ili druge grupe parkinga. Na primjer, moguće je kreirati grupu parkinga za jedan grad, a unutar nje više pojedinačnih parking objekata.